# **cgrad**: A Minimal Neural Network Library with Automatic Differentiation in C

Gregory Frommelt

Project Proposal  Programming Languages and Paradigms  University of Zurich, Fall 2026

## 1 Project Description

**cgrad** is a small, self-contained library for building and training feed-forward neural networks, written from scratch in C. It uses only the C standard library and `libm`, with no machine-learning or linear-algebra libraries. At its core is a tensor type and a computational graph that is built dynamically during the forward pass. A reverse-mode automatic differentiation engine traverses this graph backwards to compute gradients. On top of it, the library provides common differentiable operations, simple network layers and a gradient-based optimiser.

The library will be demonstrated end to end. First it learns small toy problems (e.g. XOR). Then it trains a multi-layer perceptron on a standard real-world benchmark (e.g. handwritten-digit classification on MNIST, with the dataset loaded directly from its binary file format).

## 2 Motivation: Why C?

- **Explicit lifetimes.** An autograd engine creates and discards many graph nodes in every training iteration. In garbage-collected languages this is invisible. In C, the lifetime and ownership of every node has to be designed explicitly, which makes the core engineering problem of such a system tangible.
- **Control over memory and performance.** Numerical code depends on contiguous memory, data layout and cache behaviour. C exposes all of these directly, and it is the layer on which established numerical and ML libraries are built.
- **Polymorphism by hand.** Each operation needs its own backward rule. In C this is built from structs and function pointers, which shows how dynamic dispatch works beneath object-oriented abstractions.
- **No hidden runtime.** Every allocation and every computation is explicit. This makes the system easy to reason about, to measure and to compare with high-level frameworks.

## 3 Key Language Features

The project will likely rely on the following C features:

| **Feature** | **Why** | **How** |
|---|---|---|
| Structs, `typedef`, `enum` | Model the core abstractions | Tensors, graph nodes, layers, optimiser state |
| Pointers, pointers to pointers | Represent graph edges and collections | Links from a node to its inputs; lists of nodes and parameters |
| Dynamic memory (`malloc`/`free`) | Graphs are built and destroyed at run time | Explicit ownership rules for long-lived and per-iteration objects |
| Function pointers | Per-operation gradient rules | Each node stores the backward function of the operation that created it |
| `void *` | Operation-specific saved state | Generic context attached to graph nodes |

| Feature | Why | How |
|---|---|---|
| Headers, `static`, opaque types | Encapsulation without classes | Small modules with clear public interfaces and private internals |
| Preprocessor | Build configuration and testing | Header guards, debug/release switches, assertion macros for tests |
| Binary I/O, fixed-width integers | Read datasets and possibly store models | Parsing binary file formats in a portable way |

# 4 System Architecture

The library will be split into a few loosely coupled modules, roughly:

- **Tensor core**: a two-dimensional ($n \times m$) numeric container with its data, gradient storage and basic utilities.
- **Autodiff engine**: records the computational graph, orders it topologically and runs the backward pass.
- **Operations**: differentiable building blocks such as arithmetic, matrix multiplication, activation functions and loss functions, each with a forward and a backward rule.
- **Network layers**: e.g. fully connected layers composed into a multi-layer perceptron.
- **Optimisation**: parameter updates based on the computed gradients.
- **Data handling**: dataset loading, batching and shuffling.

**Memory strategy.** Objects fall into two lifetimes. Model parameters persist across the whole training run. Intermediate results of the forward pass live for a single iteration and are released together after each update step. Clear ownership rules decide which component allocates and which one frees each kind of object.

**Tooling.** The project uses C11 and a plain `Makefile` with strict compiler warnings, a small hand-written test suite, and runtime checks for memory errors and undefined behaviour (sanitizers and leak detection).

# 5 Scope

**Core (must-haves)**

- Tensor type with a dynamically built computational graph
- Reverse-mode automatic differentiation
- A set of common differentiable operations (arithmetic, matrix operations, activations, losses)
- A multi-layer perceptron and at least one gradient-based optimiser
- Verification of the computed gradients (e.g. against numerical approximations)
- Training on a toy problem and on a real dataset

**Possible extensions (chosen depending on progress)**

- Additional optimisers
- Custom memory management for graph nodes (e.g. arena or pool allocation)
- Performance work (e.g. cache-aware kernels, multi-threading)
- Saving and loading trained models
- Comparison with an established framework in accuracy and speed

**Out of scope**

- GPU support
- Convolutional or recurrent architectures; tensors beyond two dimensions
- General-purpose broadcasting

- Bindings to other languages

# 6 Challenges and Mitigation

| **Challenge** | **Mitigation** | **Scope fallback** |
|---|---|---|
| Memory management of the graph | Ownership rules defined up front; sanitizers and leak checks from the start | Keep the simplest working allocation strategy |
| Correctness of gradients | Numerical gradient checks for every operation | Reduce the set of supported operations |
| Numerical stability | Numerically stable formulations of critical functions (e.g. softmax and logarithms) | Restrict to well-behaved operations |
| Training performance | Mini-batching, optimised builds, attention to memory layout | Train on a subset of the data or a smaller model |
| Learning curve of C | Incremental development, small test-covered steps | Drop optional extensions first |

Core functionality comes first. Extensions are only started once the core works end to end, so the project stays deliverable even if parts take longer than expected.